

Chapter 3 - Part III: Games as Search Problems

Zehara Eshetu

Department of Computer Science

2026

Outline

- ① Adversarial Search
- ② Game Search Problem
- ③ Game Search Techniques

Outline

- ① Adversarial Search
- ② Game Search Problem
- ③ Game Search Techniques

Outline

- ① Adversarial Search
- ② Game Search Problem
- ③ Game Search Techniques

Outline

- ① Adversarial Search
- ② Game Search Problem
- ③ Game Search Techniques

Introduction

- Game-playing programs developed by AI researchers since the beginning of the modern AI era (1950s)
- Games are well-defined problems that are generally interpreted as requiring intelligence to play well.
- The most common types of game problems are:
 - Adversarial games: win of one player is a loss of the other
 - Cooperative games: players have common interests and utility function
- In this course, we focus on adversarial games only
- In Adversarial games win of one player is a loss of the other.
 - one player's attempts to maximize their fitness (win) is opposed by another player.

Adversarial Search

Adversarial search is a search, where we examine the problem which arises when we try to plan ahead of the world and other agents are planning against us.

- In previous topics, we have studied the search strategies which are only associated with a single agent that aims to find the solution which often expressed in the form of a sequence of actions.
- But, there might be some situations where more than one agent is searching for the solution in the same search space, and this situation usually occurs in game playing.
- The environment with more than one agent is termed as multi-agent environment, in which each agent is an opponent of other agent and playing against each other.
 - Each agent needs to consider the action of other agent and effect of that action on their performance.
- So, Searches in which two or more players with conflicting goals are trying to explore the same search space for the solution, are called **adversarial searches**, often known as **Games**.

Typical Case

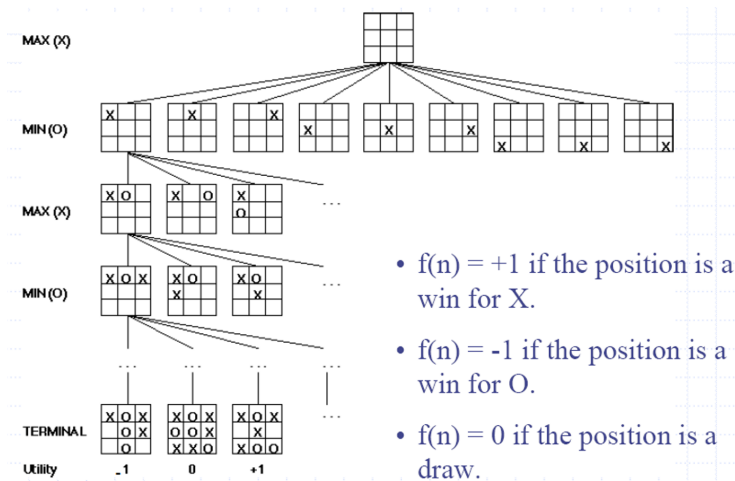
- **2-person game:** two players alternate moves
- **Zero-sum game:** one player's loss is the other's gain
 - The zero-sum assumption allows us to use a single evaluation function to describe the goodness of a board with respect to both players.
 - $f(n) > 0$: position n good for A and bad for B
 - $f(n) < 0$: position n bad for A and good for B
 - $f(n)$ near 0: position n is a neutral position
 - $f(n) = +\text{infinity}$: win for A
 - $f(n) = -\text{infinity}$: win for B
- **Perfect information:** both players have access to complete information about the state of the game.
 - No information is hidden from either player.
 - Perfect information is the equivalent to full observability with multiple agents
 - In perfect-information games, agents act sequentially and, an agent has to act, it gets to observe the state of the world before deciding what to do.

Example of an adversarial game: Tic-tac-toe

- Given $n \times n$ matrix, the objective of the game is to make three of your symbol in a row, column or diagonal
 - This is a win state for the game.
- One player is designated as player **X** and makes the first play by marking an **X** into any of the $n \times n$ open squares of the board.
 - Think a 3×3 square board where the player can put an X in any of the 9 open places
- The second player, "O", then follows suit by marking an **O** into any of the other open squares that remain.
- This continues back-and-forth until one player wins the game or players fill all squares on the board without establishing a winner (this is a draw).

Partial Game Tree for Tic-Tac-Toe

Player 1 (X) moves first



Outline

- ① Adversarial Search
- ② Game Search Problem
- ③ Game Search Techniques

Game Search Problem

- **Game problem formulation:**
 - **Initial state:** initial board position + info whose move it is
 - **Operators:** legal moves a player can make
 - **Goal (terminal test):** determines when the game is over
 - **Utility (payoff) function:** measures the outcome of the game and its desirability
- **Search objective:**
 - find the sequence of player's decisions (moves) maximizing its utility (payoff)
 - consider the opponent's moves and their utility

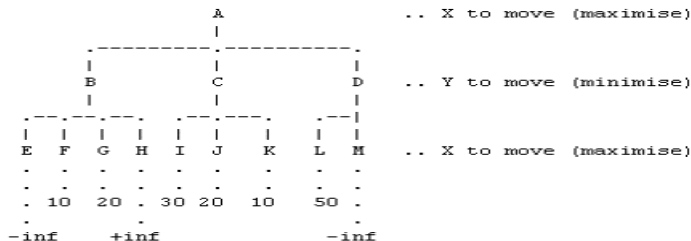
Game Tree

- A game tree is a tree where nodes of the tree are the game states and Edges of the tree are the moves by players.
- Game tree involves initial state, actions function, and result function.
- Example: Tic-Tac-Toe game tree. Following are some key points of the game:
 - There are two players MAX and MIN.
 - Players have an alternate turn and start with MAX.
 - MAX maximizes the result of the game tree
 - MIN minimizes the result.

Example Game Tree

Game between players X and Y. Let us analyze from X's perspective by looking ahead of two moves.

- H is a winning state for X (marked by plus infinity).
 - If H reached, X wins.
 - No further moves.
- E, M are losing states for X (marked by minus infinity).
 - If E or M reached, X loses.
 - No further moves.



How to go searching?

- For many games, we can look the progress of the game based on the player's position on the board.
 - We can use heuristics to evaluate these board positions and judge how good (a chance of winning) any one of the next moves can be.
 - Evaluation is done on leaf nodes
 - $+\text{inf}$ stands for sure win,
 - $-\text{inf}$ for sure loss.
 - Other numbers stand for intermediate values
- To obtain values of non-leaf nodes, analyze alternate levels in game tree as maximizing and minimizing levels.
- Two techniques:
 - **Min-max algorithm**
 - **Alpha and Beta pruning**

Outline

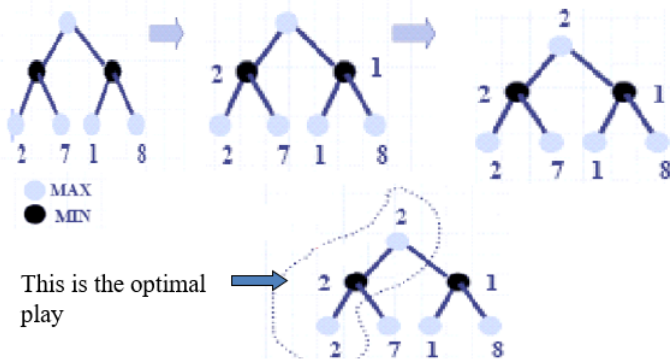
- ① Adversarial Search
- ② Game Search Problem
- ③ Game Search Techniques

Min-Max Algorithm

- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory.
- It provides an optimal move for the player assuming that opponent is also playing optimally.
- It is a method in decision theory for minimizing the maximum possible loss.
 - Alternatively, it can be thought of as maximizing the minimum gain (**max-min**).
 - It started from two player zero-sum game theory, considering the case where players take alternate moves.
- The Min-max algorithm helps find the best move, by working backwards from the end of the game.
 - At each step it assumes that player A is trying to **maximize** the chances of A winning,
 - On the next turn player B is trying to **minimize** the chances of winning (i.e., to maximize B's own chances of winning).

Example

- Two players **MAX** and **MIN** take turn (with MAX playing first)
 - Score tells whether a terminal state is a win, loss, or draw (for MAX)
- Perfect knowledge of states, no uncertainty in successor function



Min-Max Procedure

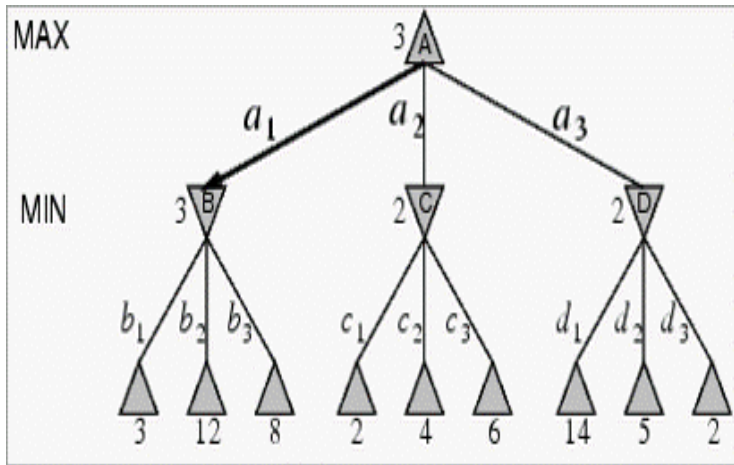
- Create start node as a MAX node with current board configuration
- Expand nodes down to some **depth** of look ahead in the game
 - The min-max algorithm proceeds with depth-first search down to the terminal states at the bottom of the tree.
- Apply the evaluation function at each of the leaf nodes
- “Back up” values for each of the non-leaf nodes until a value is computed for the root node
 - At MIN nodes, the backed-up value is the minimum of the values associated with its children.
 - At MAX nodes, the backed up value is the maximum of the values associated with its children.

Min-max algorithm

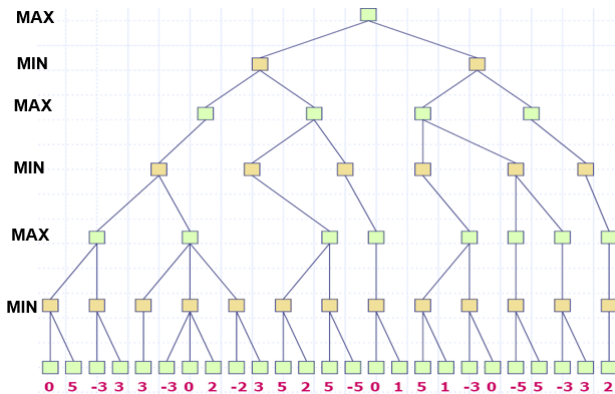
Algorithm 1 Pseudo-code for Min-Max Algorithm

```
1: function min-max(node, depth)
2:   if node is a root node or depth = MaxDepth then
3:     return the heuristic value of node
4:   if MIN turn then
5:     minimum =  $+\infty$ 
6:     for each child of node do
7:       minimum = min(minimum, min-max(child, depth+1))
8:     return minimum
9:   else
10:    maximum =  $-\infty$ 
11:    for each child of node do
12:      maximum = max(maximum, min-max(child, depth+1))
13:    return maximum
14: end min-max
```

Use Min-max to find optimal play



Exercise: Identify the optimal play path?



Properties of Mini-Max Algorithm

- **Complete**- Min-Max algorithm is Complete. It will definitely find a solution (if exist), in the finite search tree.
- **Optimal**- Min-Max algorithm is optimal if both opponents are playing optimally.
- **Time complexity**- As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is $O(b^m)$, where b is branching factor of the game-tree, and m is the maximum depth of the tree.
- **Space Complexity**- Space complexity of Mini-max algorithm is also similar to DFS which is $O(bm)$.

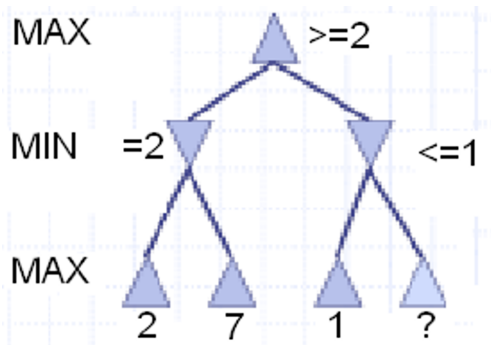
Limitation of the minimax Algorithm

- The main drawback of the minimax algorithm is that it gets really slow for complex games such as chess, etc.
- This type of games has a huge branching factor, and the player has lots of choices to decide.
- This limitation of the minimax algorithm can be improved from alpha-beta pruning which we have discussed in the next slides.

Alpha-Beta(α - β) Pruning

- We can improve on the performance of the min-max algorithm through alpha-beta pruning
- Idea is that we can find minimax solution without looking at every node in game tree
- We “**prune**” unnecessary parts of the game tree (i.e. remove them)
- Always returns the same move as standard minimax, but prunes away branches that cannot influence final decision
- Alpha-Beta pruning reduces the complexity by pruning parts of the tree that won't influence decision making
 - With this we don't have to search the complete tree.

Alpha-Beta(α - β) Pruning

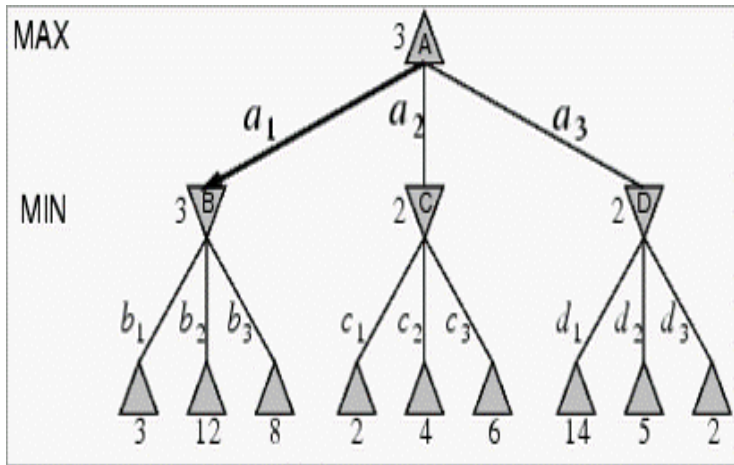


- We don't need to compute the value at this node
- No matter what it is, it can't affect the value of the root node

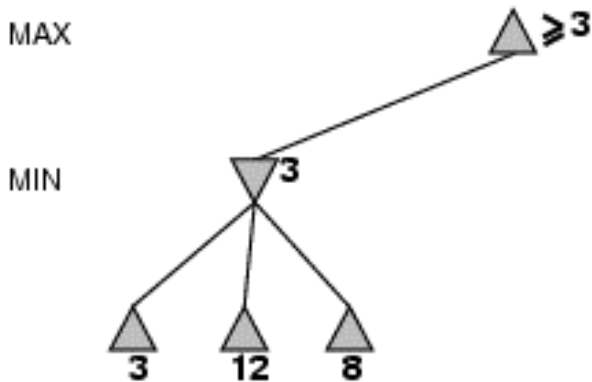
Alpha-Beta Procedure

- Two values are used by this procedure.
 - **Alpha** - the highest-value found so far at any choice-point along the path for Max
 - **Beta** - the lowest-value found so far at any choice-point along the path for Min
- Traverse the search tree in depth-first order and update alpha and beta values for each search node as the search progresses
 - At each MAX node n , **alpha(n)** = maximum value found so far
 - At each MIN node n , **beta(n)** = minimum value found so far
 - The alpha values start at $-\infty$ and only increase, while beta values start at $+\infty$ and only decrease.
- Prune branches that can not change the final decision
 - Stop searching when $\alpha(i) \geq \beta(n)$ for some MAX node ancestor i of n .

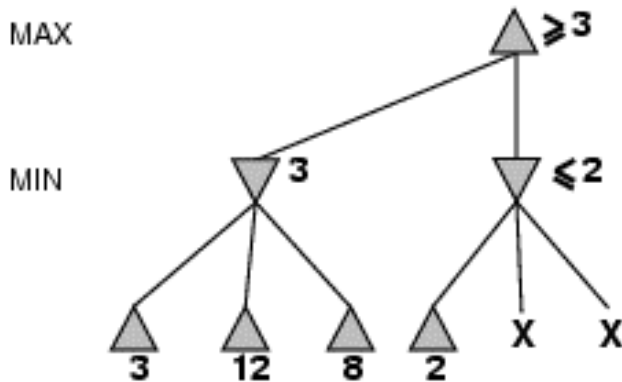
Example



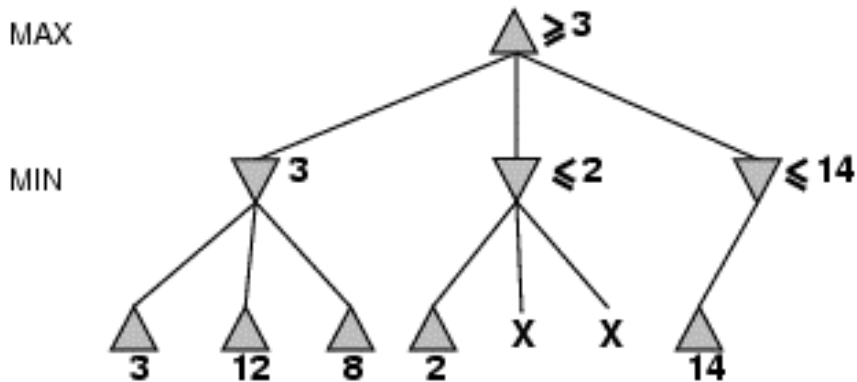
α - β Pruning Example (1)



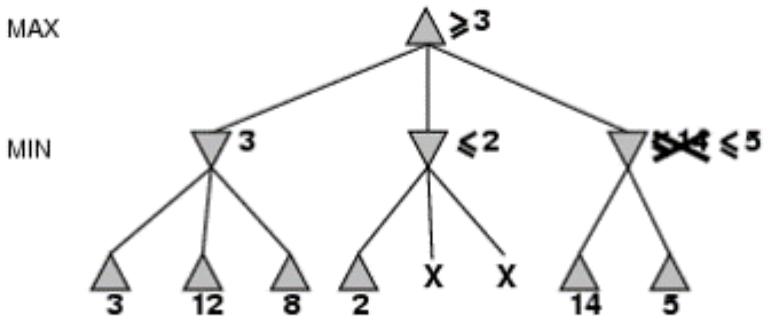
α - β Pruning Example (2)



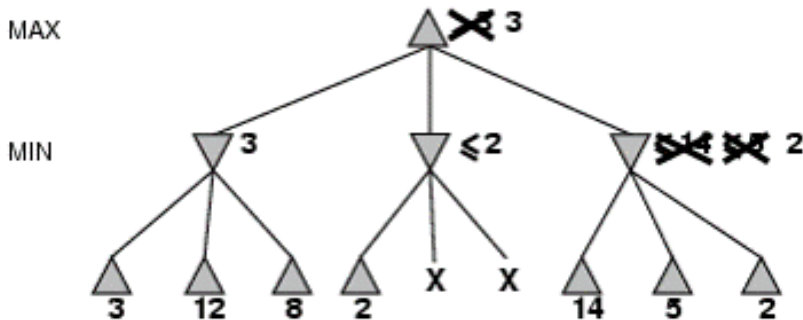
α - β Pruning Example (3)



α - β Pruning Example (4)



α - β Pruning Example (5)



α - β Pruning

- α = the value of the best (i.e. highest) choice we have found so far at any choice point along the path for max
- β = the value of the best (i.e. lowest) choice we have found so far at any choice point along the path for min

Effectiveness of Alpha-Beta

- **Alpha-beta** is guaranteed to compute the same value for the root node as computed by min-max, with less or equal computation
- **Worst case:** no pruning, examining bd leaf nodes, where each node has b children and a d -depth search is performed
- **Best case** is when each player's best move is the first alternative generated
 - **Best case:** examine only $bd/2$ leaf nodes.
- The result shows that you can search twice as deep as min-max.

[illegible]